

Proyecto Final: Mapa meteorológico

DISEÑO ARQUITECTÓNICOS Y PATRONES.

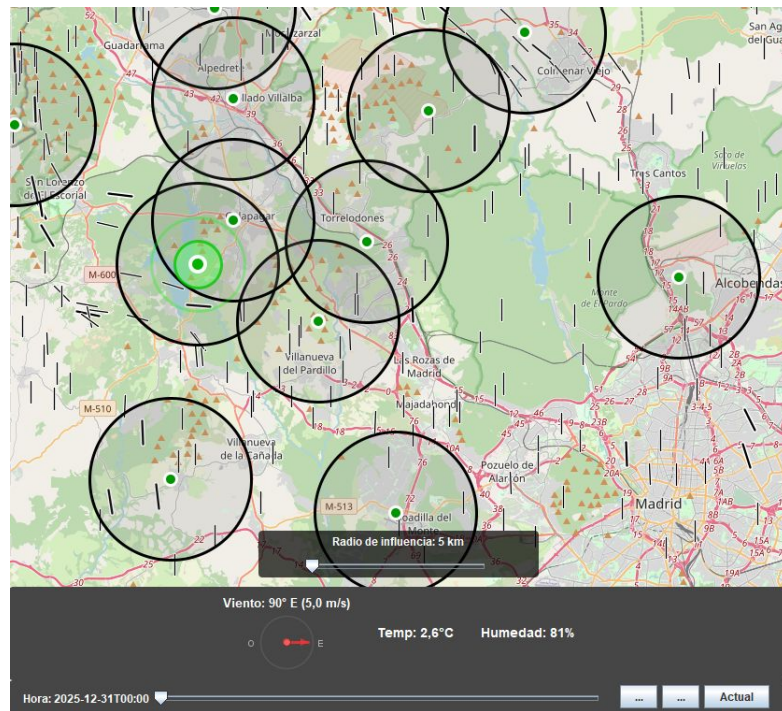
Héctor Luís Mariño Fernández.

Descripción del Proyecto

— — —

Meteorologic-Map es una aplicación robusta desarrollada en **Java 11** que permite la visualización interactiva de datos climáticos sobre mapas de **OpenStreetMap**. El proyecto nace con el objetivo de integrar fuentes de datos meteorológicos reales (Datos de estaciones meteorológicas en tiempo real) en una interfaz gráfica (Swing) que sea tanto fluida como altamente mantenible.

La arquitectura ha sido diseñada deliberadamente utilizando **8 patrones de diseño** para garantizar que el sistema pueda crecer, cambiar de proveedores de datos o añadir nuevas visualizaciones sin degradar su rendimiento ni complicar su código base.



Funcionalidad Implementada

— — —

La aplicación ofrece una experiencia de usuario centrada en la exploración geográfica y temporal:

- **Exploración Dinámica:** Al desplazar el mapa, el sistema identifica automáticamente la **estación meteorológica más cercana** al centro de la vista para mostrar sus metadatos y lecturas.
- **Gestión de Datos en Tiempo Real:** Los datos de temperatura, humedad y viento se normalizan a unidades estándar (Celsius, m/s) independientemente de la fuente externa.
- **Capas de Visualización (Overlays):** Permite activar capas específicas de **temperatura o humedad**, garantizando que solo una esté activa a la vez para evitar el desorden visual.
- **Barra Temporal Global:** Un control persistente que permite ajustar el instante meteorológico. Lo más destacado es que **mantiene el tiempo seleccionado** incluso si el usuario cambia de estación o mueve el mapa.
- **Optimización de Partículas:** Visualiza el movimiento del viento mediante miles de partículas renderizadas de forma eficiente a **30 FPS**.

Datos Meteorológicos

```
{
  "latitude" : 40.52724,
  "longitude" : -4.0677795,
  "generationtime_ms" : 3.200650215148926,
  "utc_offset_seconds" : 3600,
  "timezone" : "Europe/Madrid",
  "timezone_abbreviation" : "GMT+1",
  "elevation" : 853.0,
  "hourly_units" : {
    "time" : "iso8601",
    "temperature_2m" : "°C",
    "relative_humidity_2m" : "%",
    "wind_speed_10m" : "km/h",
    "wind_direction_10m" : "°"
  },
  "hourly" : {
    "time" : [ "2025-12-31T00:00", "2025-12-31T01:00", "2025-12-31T02:00", "2025-12-31T03:00" ],
    "temperature_2m" : [ 2.6, 2.0, 1.4, 0.3 ],
    "relative_humidity_2m" : [ 81, 82, 86, 88 ],
    "wind_speed_10m" : [ 10.2, 8.2, 5.4, 5.1 ],
    "wind_direction_10m" : [ 330, 327, 330, 330 ]
  }
}
```

Patrones de Diseño Utilizados

Este es el pilar técnico de la aplicación. Se han implementado los siguientes patrones para resolver retos específicos:

- **MVP (Model-View-Presenter):** Separa la interfaz (`WeatherMapWindow`) de la lógica de negocio (`WeatherService`) mediante un presentador (`WeatherMapPresenter`). Esto permite probar la lógica de la aplicación sin necesidad de abrir la ventana gráfica.
- **Adapter:** Actúa como traductor entre las APIs externas (Open-Meteo, Meteostat) y el contrato interno del sistema. Gracias a esto, si una API cambia su formato de datos, solo es necesario modificar su adaptador.
- **Decorator:** Implementado en la clase `CachingWeatherClient`, añade una capa de **almacenamiento en caché** de forma transparente. Mejora el rendimiento al evitar peticiones repetidas a Internet para los mismos datos.
- **Factory:** Centraliza la creación de objetos complejos. Las fábricas (`WeatherClientFactory`, `ProviderFactory`) deciden qué adaptadores o decoradores aplicar según la configuración, ocultando esa complejidad al resto del código.

Patrones de Diseño Utilizados (2)

Strategy (Overlays): Cada capa de visualización es una estrategia independiente. Esto permite añadir una nueva capa (por ejemplo, de presión atmosférica) simplemente creando una clase que cumpla el contrato `WeatherOverlay`, sin tocar el código del mapa.

Flyweight (Peso Ligero): Crucial para el rendimiento. Reutiliza estilos de marcadores y partículas de viento en lugar de crear miles de objetos idénticos. Agrupa el viento en **8 sectores cardinales** y rangos de velocidad para compartir un número reducido de instancias en memoria.

Observer (Event Bus): Desacopla los componentes mediante un sistema de publicación y suscripción. Por ejemplo, cuando se cargan las estaciones, el `EventBus` notifica a todos los interesados (paneles, mapa) sin que estos tengan que conocerse entre sí.

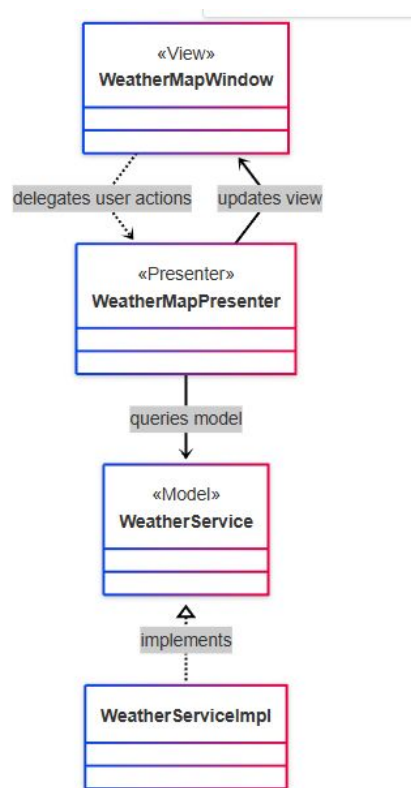
State (con Gestor): El `OverlayManager` controla qué capa está activa. Funciona como un gestor de estados que garantiza la **exclusividad**, desactivando automáticamente la capa de temperatura si el usuario activa la de humedad.

Patrón MVP (Model-View-Presenter)

El objetivo primordial de este patrón es **desacoplar la interfaz de usuario de la lógica de presentación y de los modelos de datos**, mejorando la testabilidad y mantenibilidad.

- **Componentes:** La **View** está representada por `WeatherMapWindow`, encargada de renderizar el mapa y delegar acciones del usuario. El **Presenter**, encarnado en `WeatherMapPresenter`, actúa como mediador coordinando datos y actualizaciones. El **Model** (`WeatherService`) proporciona los datos y la lógica de negocio.

- **Beneficios:** Permite probar el presentador de forma aislada sin una interfaz real y facilita la migración de tecnologías de UI (como pasar de Swing a otro framework) sin afectar la lógica subyacente.



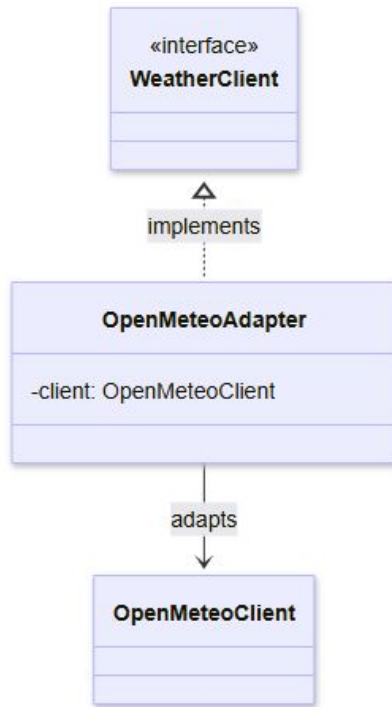
Patrón Adapter (Adaptador)

— — —

Se utiliza para asegurar la **interoperabilidad entre interfaces incompatibles**, específicamente para unificar distintos proveedores de datos externos bajo un mismo contrato.

- **Funcionamiento:** `OpenMeteoAdapter` implementa la interfaz interna `WeatherClient`, traduciendo las peticiones genéricas del sistema al formato específico que requiere la API de Open-Meteo.
- **Beneficios:** El núcleo de la aplicación queda aislado de cambios en las APIs externas. Esto permite **sustituir un proveedor por otro** simplemente creando un nuevo adaptador sin modificar el resto del sistema.

Adapter Pattern (WeatherClient)

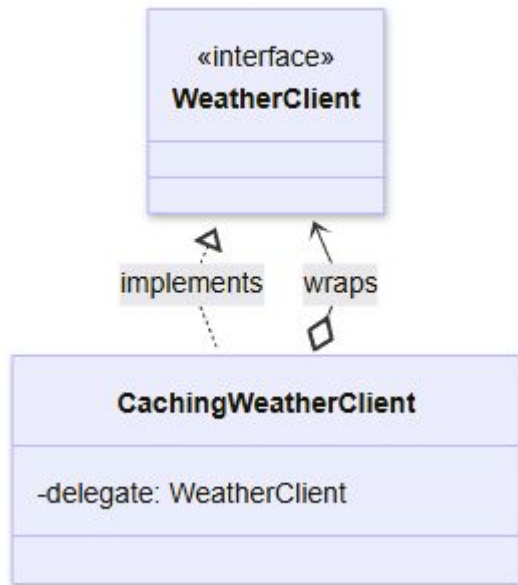


Patrón Decorator (Decorador)

Este patrón permite **añadir responsabilidades a un objeto de forma dinámica** sin alterar su clase base, utilizado aquí para implementar la gestión de caché.

- **Implementación:** La clase `CachingWeatherClient` implementa `WeatherClient` y envuelve a otra instancia del mismo tipo (el delegado).
- **Beneficios:** Añade una capa de **almacenamiento en caché transparente** que intercepta las solicitudes, mejorando el rendimiento percibido y mitigando problemas de límites de uso (rate limiting) de las APIs externas.

Decorator Pattern (CachingWeatherClient)

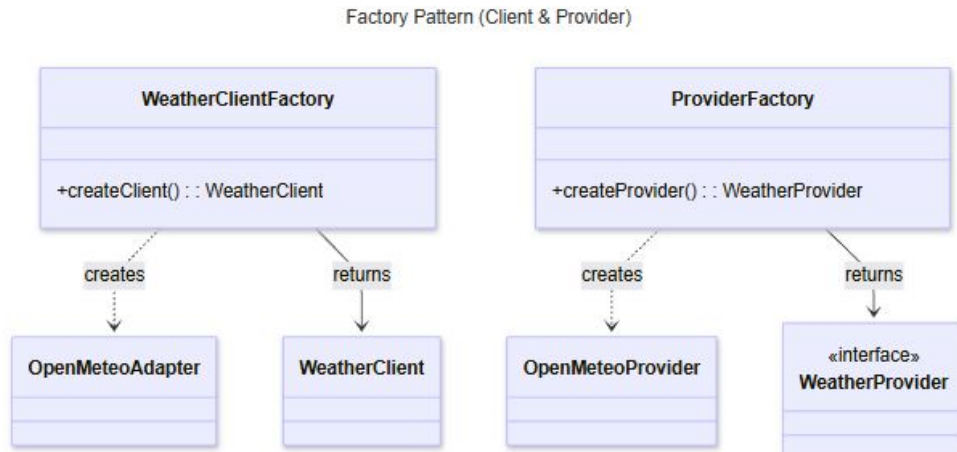


Patrón Factory (Fábrica)

Centraliza y encapsula la **lógica de creación de objetos complejos**, ocultando los detalles de cómo se componen los adaptadores y decoradores.

- **Fábricas clave:** **WeatherClientFactory** construye instancias completas de clientes configurados, mientras que **ProviderFactory** genera los proveedores de datos de alto nivel.

- **Beneficios:** Proporciona un **único punto de ensamblaje**, lo que facilita cambios de configuración globales y garantiza que todos los objetos se instancian de manera coherente en todo el sistema.



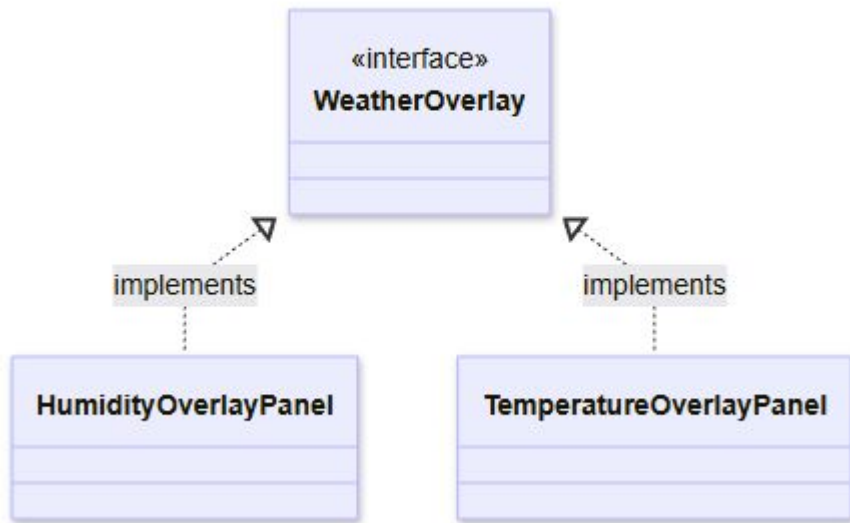
Patrón Strategy

Define una familia de **algoritmos de renderizado intercambiables**, permitiendo seleccionar diferentes capas visuales en tiempo de ejecución.

- **Componentes:** La interfaz `WeatherOverlay` define el contrato común, mientras que `TemperatureOverlayPanel` y `HumidityOverlayPanel` son las estrategias concretas para pintar datos en el mapa.

- **Beneficios:** Cumple el principio de **Abierto/Cerrado**, permitiendo añadir nuevas visualizaciones (como presión o viento) simplemente creando una nueva clase sin modificar el código existente.

Overlay Pattern (WeatherOverlay Panels)



Patrón Flyweight (Peso Ligero)

— — —

Su función es **optimizar el uso de memoria** compartiendo estados inmutables entre un gran número de objetos, lo cual es crítico para el rendimiento del mapa.

- **Aplicación:** `WeatherFlyweightFactory` gestiona estilos compartidos para marcadores y partículas de viento. Discretiza la dirección del viento en **8 sectores cardinales** y la velocidad en rangos específicos ("buckets").

- **Beneficios:** Reduce drásticamente la creación de objetos repetitivos, lo que **disminuye la asignación de memoria** y acelera el renderizado de miles de partículas de viento simultáneas.

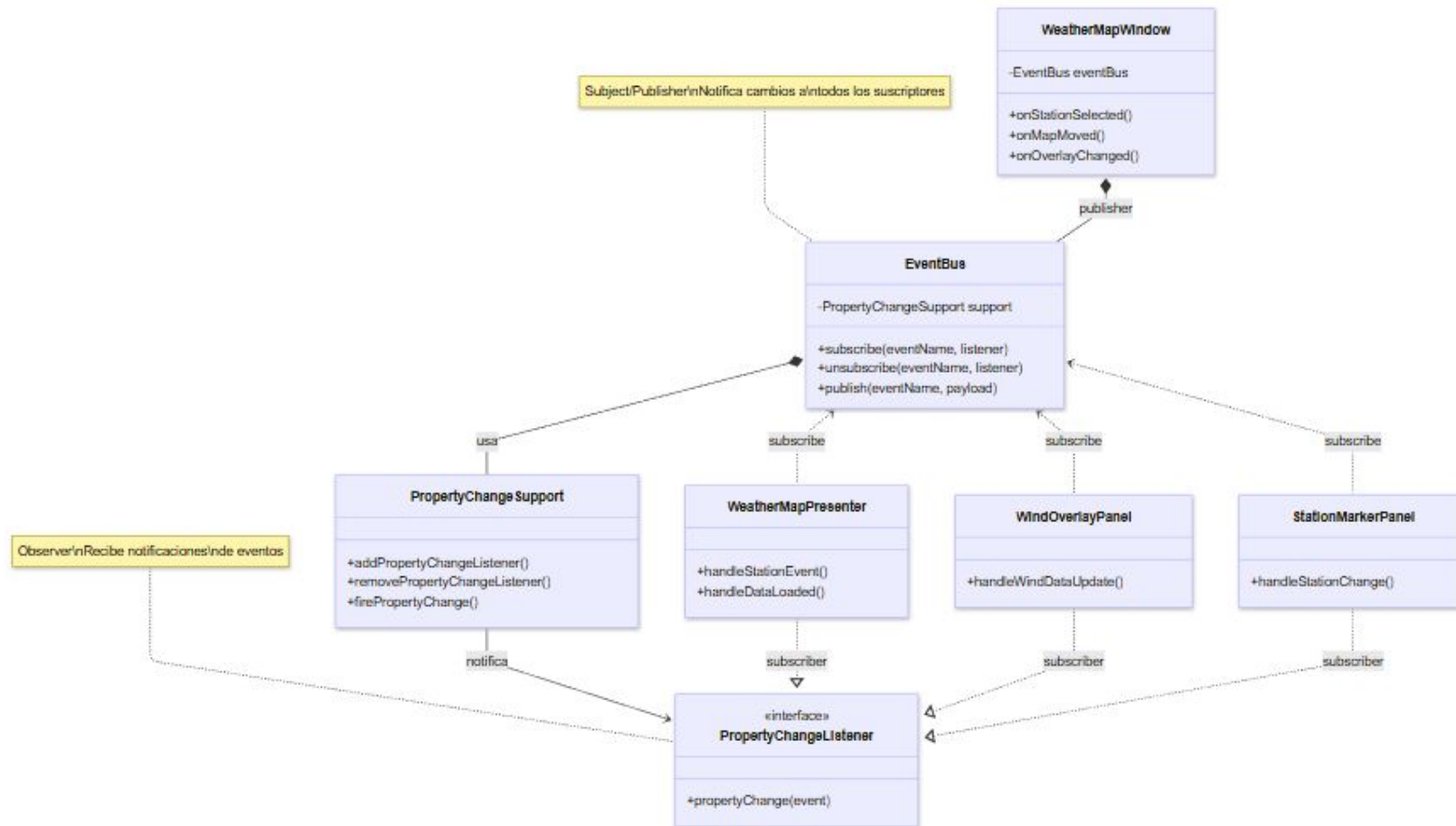
Flyweight Pattern (Marker & Particle Styles)



Patrón Observer

Establece un modelo de comunicación de **publicación-suscripción** que desacopla totalmente a los emisores de eventos de sus receptores.

- **Funcionamiento:** La clase `EventBus` gestiona el registro de suscriptores y la difusión de eventos como `StationsLoaded` (estaciones cargadas) o `WeatherSnapshotUpdated`.
- **Beneficios:** Mejora la **extensibilidad**, permitiendo que nuevos componentes reaccionen a cambios de estado del sistema sin que el emisor original tenga conocimiento de su existencia.



Patrón State

Se emplea para controlar el estado de la aplicación y garantizar que se cumplan las reglas de exclusividad de la interfaz.

- **Gestión:** `OverlayManager` utiliza la enumeración `OverlayMode` para asegurar que **solo una capa visual esté activa** a la vez.
- **Beneficios:** Centraliza las transiciones de estado, simplificando el código de la interfaz y **previniendo estados inconsistentes**, como el solapamiento accidental de capas de temperatura y humedad.

Patrón State - OverlayManager

Contexto\nGestiona el estado activo\ny delega comportamiento\nal overlay correspondiente

OverlayManager

-Map<OverlayMode, WeatherOverlay> overlays
-OverlayMode activeMode

+register(mode, overlay)
+setActiveMode(mode)
+getActiveMode(): OverlayMode

estado actual

State\nDefine los posibles\nestados del sistema

«enumeration»
OverlayMode

NONE
HUMIDITY
TEMPERATURE

State Interface\nDefine operaciones\nque varían según estado

«interface»
WeatherOverlay

+setActive(active)
+isActive(): bool
+setStations(stations)
+setInfluenceRadiusKm(radiusKm)
+onMapChanged()

gestiona

implements

HumidityOverlayPanel

-bool active
-List<WeatherStation> stations

+setActive(active)
+isActive(): bool
+paintComponent(g)

implements

TemperatureOverlayPanel

-bool active
-List<WeatherStation> stations

+setActive(active)
+isActive(): bool
+paintComponent(g)

implements

WindOverlayPanel

-bool active
-List<WindParticle> particles

+setActive(active)
+isActive(): bool
+paintComponent(g)

Problemas Enfrentados y cómo se solucionaron

Durante el desarrollo, se superaron varios obstáculos críticos:

- **Rate Limiting (Error 429):** Las APIs externas limitan la frecuencia de peticiones. Se soluciona mediante el **patrón Decorator** (caché).
- **Presión sobre el Garbage Collector:** Renderizar miles de partículas generaba latencia. Se resolvió mediante **Object Pooling** (reutilización de objetos) y el patrón **Flyweight**, reduciendo la creación de objetos en un 70%.
- **Latencia en la UI:** Las llamadas a APIs externas bloqueaban la interfaz. Se implementó un **AsyncExecutor** para gestionar tareas en hilos secundarios, manteniendo el mapa fluido durante las cargas.

Posibles mejoras futuras

- **Configuración Dinámica:** Permitir al usuario cambiar el proveedor de datos (ej. de Open-Meteo a Meteostat) directamente desde la interfaz en tiempo de ejecución, aprovechando la logica del patrón Adapter.
- **Nuevas Capas:** Implementar estrategias adicionales para visualizar la presión atmosférica o alertas de precipitaciones utilizando el patrón Strategy.
- **Motor de prediccion:** Implementar llamadas a API's de meteorologia para predecir evolución futura. Actualmente solo presentamos un histórico.

Demostración del uso del programa.

— — —